

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ «МИСИС»

*ИНСТИТУТ Информационных компьютерных наук (ИКН)
КАФЕДРА Автоматизированных систем управления (АСУ)
НАПРАВЛЕНИЕ 09.03.00 ГРУППА БИВТ-24-4*

КУРСОВОЙ ПРОЕКТ

ПО КУРСУ Клиент-серверные приложения.

ТЕМА: Разработка web-приложения для учета результатов тестов (Test Results)

Студент Елдашов В.Э.



*Проверил: Абросимов Н. А.
Шелудяков П. А.
Микитенко И.И.*

Оценка выполнения курсового проекта _____

Москва 2026 г.

Оглавление

Постановка задачи	4
Функциональные требования	4
Нефункциональные требования	4
Описание архитектуры.....	5
Уровень представления	5
Уровень бизнес-логики	5
Уровень хранения данных	6
Описание структуры базы данных.....	7
Модель	7
Таблицы	7
Представления.....	9
Триггеры	9
Описание серверной части.....	11
Каталог routers.....	11
Каталог schemas	12
Каталог service	12
Логирование	13
Безопасность.....	14
Описание клиентской части.....	15
Заключение.....	21
Список литературы.....	22

Предметная область

Результаты тестирования являются одним из основных источников информации о том, насколько студент освоил материал. В учебном процессе такие данные используются не только для выставления оценки, но и для поиска слабых тем, планирования повторения и контроля динамики успеваемости.

Если результаты хранятся в отдельных файлах или пересылаются вручную, быстро возникают типичные проблемы: сложно понять, какая попытка была последней, кто уже прошел тест, где был изменен балл и почему итоговая оценка отличается от ожидаемой. Особенно заметно это становится, когда тестов несколько, а студентов в группе много.

В рамках курсовой работы рассматривается небольшая система учета результатов тестов. Она хранит пользователей, тесты, вопросы, варианты ответов и итоговые баллы. Преподаватель получает инструмент для фиксации и просмотра результатов, а студент – доступ к собственным оценкам.

Основные проблемы предметной области:

- отсутствие единого структурированного места для хранения тестов и результатов;
- риск ручных ошибок при подсчете процента и оценки;
- необходимость разграничить доступ между студентом, преподавателем и администратором;
- потребность видеть сводку по тестам без ручной обработки таблиц.

Решением является клиент-серверное web-приложение, в котором клиент отвечает за интерфейс, сервер – за проверку данных и API, а база данных – за надежное хранение структуры тестирования.

Постановка задачи

Вариант 7. Тема: Результат теста.

Целью данной курсовой работы является проектирование и реализация клиент-серверного приложения Test Results, обеспечивающего учет тестов, пользователей и результатов прохождения. Приложение должно поддерживать хранение тестов, управление результатами, ролевой доступ и обмен данными через REST API [2].

Функциональные требования

- приложение должно обеспечивать просмотр списка тестов и результатов тестирования;
- приложение должно поддерживать создание, редактирование и удаление тестов преподавателем или администратором;
- приложение должно поддерживать создание, редактирование и удаление результатов прохождения тестов;
- приложение должно поддерживать создание пользователей через обязательный маршрут POST /users;
- приложение должно выполнять авторизацию пользователя по email и паролю с использованием cookie-сессии на стороне сервера;
- приложение должно автоматически вычислять процент выполнения, итоговую оценку и статус зачета;
- приложение должно отображать агрегированную статистику: средний процент, число результатов и число зачтенных попыток;
- приложение должно разграничивать доступ между ролями student, teacher и admin.

Нефункциональные требования

- клиентская часть реализуется на React с использованием Vite;
- серверная часть реализуется на Python с использованием FastAPI;
- в составе курсовой работы разрабатывается схема PostgreSQL, содержащая таблицы, представления, функции, процедуры и триггеры;
- обмен между клиентом и сервером выполняется через REST API в формате JSON;
- API использует данные в оперативной памяти и сбрасывает их после перезапуска сервера, как того требует задание;

Описание архитектуры

Приложение Test Results построено по трехслойной клиент-серверной архитектуре. Уровень представления реализован на React, уровень бизнес-логики – на FastAPI, уровень хранения данных в работающем приложении – в памяти сервера. Такое разделение позволяет не смешивать интерфейс, проверку данных и обработку пользовательских сценариев.

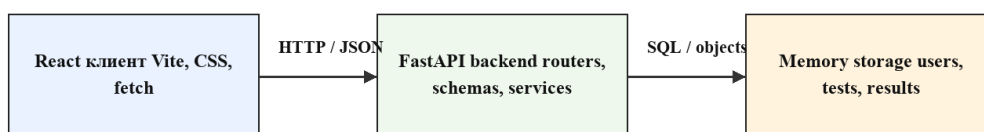


Рисунок 1 – Архитектура клиент-серверного приложения Test Results

Уровень представления

Клиентская часть представляет собой React-приложение, работающее в браузере пользователя. Интерфейс содержит рабочую область со сводкой, список тестов, таблицу результатов, форму создания пользователя и формы редактирования данных. Все данные загружаются асинхронно через Fetch API после обращения к серверу.

На уровне представления сосредоточены задачи отображения данных и организации пользовательских сценариев. Клиент отвечает за навигацию по рабочим блокам, отображение ошибок, скрывание недоступных действий для роли student и сохранение последнего введенного email в LocalStorage.

Уровень бизнес-логики

Серверная часть разделена на каталоги routers, schemas, services и вспомогательные модули dependencies, auth и storage. Роутеры принимают HTTP-запросы, Pydantic-схемы проверяют входные данные, сервисы выполняют бизнес-правила, а storage управляет

данными в памяти процесса. Такой подход позволяет отделить обработку HTTP от правил предметной области.

На уровне бизнес-логики сосредоточены проверка обязательных полей, проверка формата email, вычисление допустимых значений баллов, маршрутизация по ролям и формирование единообразных JSON-ответов. Здесь же находятся механизмы авторизации и логирование событий входа.

Уровень хранения данных

В работающей версии приложения основное хранилище реализовано в памяти backend. После запуска FastAPI подготавливает демонстрационные списки пользователей, тестов и результатов, а после перезапуска сервера эти данные возвращаются к начальному состоянию.

Отдельно в папке DB разработана реляционная схема PostgreSQL. Она не участвует в жизни приложения, но закрывает требования курсовой по проектированию таблиц, представлений, функций, процедур и триггеров.

Последовательность работы: пользователь выполняет действие в браузере, компонент React формирует HTTP-запрос, FastAPI-роутер получает данные и передает их в сервис, после чего сервис обращается к хранилищу в памяти. В ответ клиент получает JSON. LocalStorage на клиенте используется для вспомогательного хранения последнего введенного email.

Описание структуры базы данных

Модель

База данных приложения включает шесть основных сущностей: User, Test, Question, Answer, TestResult и ResultAudit. Пользователь может иметь роль student, teacher или admin. Тест объединяет вопросы, вопрос содержит варианты ответов, а результат связывает пользователя с конкретным тестом.

Один пользователь может иметь множество результатов прохождения, один тест может включать множество вопросов, а один вопрос – несколько вариантов ответа. Таблица result_audit хранит историю изменений результатов и позволяет восстановить факт корректировки баллов.

При удалении теста должны быть удалены связанные с ним вопросы и производные данные, а при изменении результата должны быть пересчитаны вычисляемые поля. Такая модель отражает предметную область учебного тестирования и позволяет продемонстрировать основные виды связей реляционной БД.

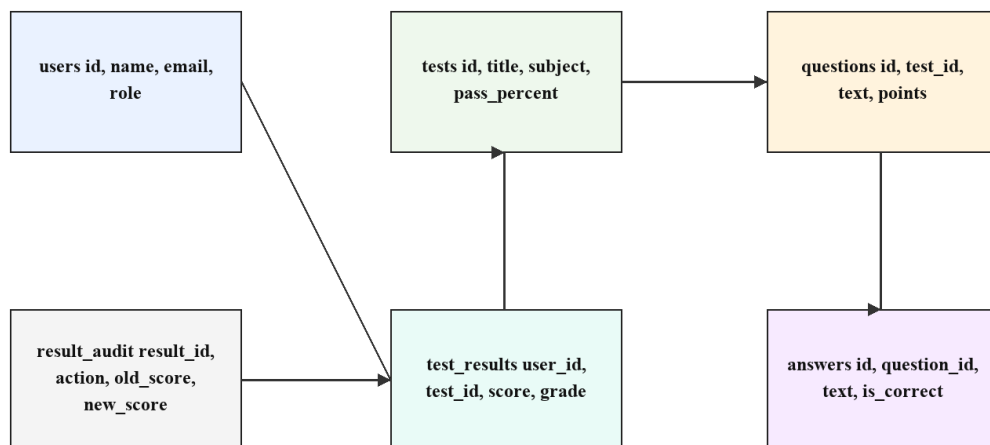


Рисунок 2 – Схема базы данных

Таблицы

В таблице 1 представлено описание таблицы users.

Таблица 1 – Структура таблицы users

Поле	Тип	Описание
id	SERIAL PK	Уникальный идентификатор пользователя

name	VARCHAR NOT NULL	Имя пользователя
email	VARCHAR UNIQUE	Электронная почта
password_hash	VARCHAR NOT NULL	Хеш пароля пользователя
age	INTEGER	Возраст пользователя
role	VARCHAR	Роль: student, teacher или admin

В таблице 2 представлено описание таблицы tests.

Таблица 2 – Структура таблицы tests

Поле	Тип	Описание
id	SERIAL PK	Уникальный идентификатор теста
title	VARCHAR NOT NULL	Название теста
subject	VARCHAR NOT NULL	Учебный предмет
description	TEXT	Описание содержания теста
pass_percent	INTEGER	Проходной процент
created_by	INTEGER FK	Пользователь, создавший тест

В таблице 3 представлено описание таблицы questions.

Таблица 3 – Структура таблицы questions

Поле	Тип	Описание
id	SERIAL PK	Уникальный идентификатор вопроса
test_id	INTEGER FK	Ссылка на тест
text	TEXT NOT NULL	Текст вопроса
points	INTEGER	Количество баллов за вопрос
created_at	TIMESTAMP	Дата создания
updated_at	TIMESTAMP	Дата изменения

В таблице 4 представлено описание таблицы answers.

Таблица 4 – Структура таблицы answers

Поле	Тип	Описание
id	SERIAL PK	Уникальный идентификатор ответа
question_id	INTEGER FK	Ссылка на вопрос
text	TEXT NOT NULL	Текст варианта ответа
is_correct	BOOLEAN	Признак правильного ответа

В таблице 5 представлено описание таблицы test_results.

Таблица 5 – Структура таблицы test_results

Поле	Тип	Описание
id	SERIAL PK	Уникальный идентификатор результата

user_id	INTEGER FK	Ссылка на пользователя
test_id	INTEGER FK	Ссылка на тест
score	INTEGER	Набранные баллы
max_score	INTEGER	Максимальный балл
percent	NUMERIC	Процент выполнения
grade	INTEGER	Итоговая оценка
passed	BOOLEAN	Статус зачета
comment	TEXT	Комментарий преподавателя

В таблице 6 представлено описание таблицы result_audit.

Таблица 6 – Структура таблицы result_audit

Поле	Тип	Описание
id	SERIAL PK	Уникальный идентификатор записи аудита
result_id	INTEGER FK	Ссылка на измененный результат
action	VARCHAR	Тип операции: insert, update, delete
old_score	INTEGER	Старое значение балла
new_score	INTEGER	Новое значение балла
changed_at	TIMESTAMP	Дата и время изменения

Представления

В базе данных определены три представления, используемые для агрегации часто запрашиваемых данных:

- v_result_summary – объединяет пользователей, тесты и результаты, возвращает имя студента, название теста, баллы, процент, оценку и статус;
- v_test_average – вычисляет средний процент, среднюю оценку и число попыток по каждому тесту;
- v_recent_results – возвращает последние результаты для вывода на сводных экранах.

Триггеры

В базе данных определены три триггера:

- trg_fill_result_fields – срабатывает перед вставкой или обновлением результата и вычисляет percent, grade и passed;
- trg_results_audit – срабатывает после изменения результата и записывает старые и новые значения в таблицу result_audit;
- trg_update_timestamp – обновляет поле updated_at при изменении записи questions или tests.

Отдельно в SQL-скрипте присутствуют функции и процедуры, отвечающие за расчет процента, перевод процента в оценку, проверку проходного балла, создание теста и очистку демонстрационных данных. Они не вынесены в отдельный подпункт, чтобы сохранить тот же каркас содержания, что и в референсном отчете.

Наличие представлений, функций, процедур и триггеров показывает, что часть логики может быть перенесена на уровень базы данных.

Описание серверной части

Серверная часть приложения реализована на Python с использованием FastAPI и организована в виде набора каталогов, каждый из которых отвечает за собственный уровень архитектуры. Точка входа – файл `main.py`, который импортирует объект `app` из модуля `app/app.py`, где настраиваются `middleware`, обработчики ошибок и подключение роутеров.

Такая архитектура упрощает поддержание кода. Необходимо разделять точку входа, HTTP-слой, бизнес-логику и работу с данными, чтобы изменения в одной части не требовали переписывать остальные модули.

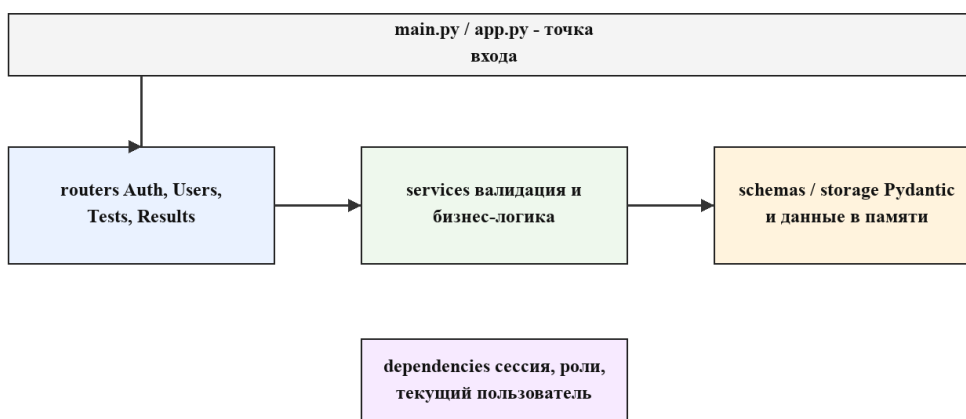


Рисунок 3 – Диаграмма серверной части

Каталог routers

Каталог `routers` содержит маршруты API. В проекте выделены отдельные роутеры для авторизации, пользователей, тестов и результатов:

- `auth.py`
- `users.py`
- `tests.py`
- `results.py`

Каждый роутер отвечает только за прием HTTP-запроса и передачу данных дальше в сервисный слой. Внутри роутеров не хранится бизнес-логика, поэтому их код остается коротким и читаемым.

Таблица 7 – Описание REST API

Метод	Маршрут	Описание
POST	<code>/login</code>	Вход в систему
POST	<code>/logout</code>	Выход из системы
GET	<code>/me</code>	Текущий пользователь
GET	<code>/users</code>	Список пользователей

GET	/users/{id}	Пользователь по идентификатору
POST	/users	Создание пользователя
GET	/tests	Список тестов
POST	/tests	Создание теста
PUT	/tests/{id}	Обновление теста
DELETE	/tests/{id}	Удаление теста
GET	/results	Список результатов
POST	/results	Создание результата
PUT	/results/{id}	Обновление результата
DELETE	/results/{id}	Удаление результата

Таким образом, серверная часть покрывает как обязательный API из задания, так и функциональность, необходимую для предметной области.

Каталог schemas

Каталог schemas содержит Pydantic-схемы, которые используются для валидации входных данных. В проекте выделены отдельные схемы для авторизации, пользователей, тестов и результатов.

Схемы решают две задачи. Они задают понятную структуру входного запроса, а также они позволяют сразу отклонять некорректные данные, например пустые строки, некорректный email, недопустимый возраст или неверные числовые значения. За счет этого значительная часть базовой проверки выполняется до перехода к бизнес-логике.

Каталог service

Каталог services содержит основную бизнес-логику приложения. Для каждой группы сущностей выделен отдельный сервис:

- AuthService
- UserService
- TestService
- ResultService

AuthService отвечает за вход в систему, завершение сессии и получение текущего пользователя. UserService работает со списком пользователей и созданием новой учетной записи. TestService реализует операции просмотра, создания, изменения и удаления тестов. ResultService выполняет операции над результатами тестирования и дополнительно контролирует связанные ограничения, например существование пользователя и теста, а также корректность баллов.

Такое разделение удобно тем, что правила предметной области сосредоточены в одном слое, а не размазаны по разным слоям.

Модуль storage

Роль хранилища в текущей версии проекта выполняет модуль storage.py. В нем реализован класс MemoryStorage, который хранит данные приложения в памяти процесса. После запуска сервера в память загружается демонстрационный набор пользователей, тестов и результатов. После перезапуска сервера состояние сбрасывается к начальному.

MemoryStorage предоставляет простые методы для работы с данными:

- получение списка пользователей, тестов и результатов;
- получение записи по идентификатору;
- создание новой записи;
- обновление существующей записи;
- удаление записи.

Отдельно в этом же модуле находится функция логирование событий авторизации. Она записывает в лог успешные входы, неудачные попытки входа и выход пользователя из системы.

Логирование

В приложении предусмотрено логирование событий авторизации. При успешном входе записывается SUCCESS_LOGIN, при ошибке входа – FAIL_LOGIN, при выходе – LOGOUT. Лог хранится в файле storage/auth.log и позволяет отследить IP-адрес и исход попытки входа.

Логирование фиксирует как успешные попытки, так и неуспешные. Это полезно не только для отладки, но и для подтверждения корректной маршрутизации и наличия валидации на уровне бизнес-логики.

```
if user is None or not verify_password(payload.password,
user["password_hash"]):
    raise HTTPException(status_code=401, detail="Неверный email или пароль")

request.session["user_id"] = int(user["id"])
```

Рисунок 4 – Логирование и проверка авторизации

Безопасность

Защита в приложении реализована на нескольких уровнях: хранение учетных данных, аутентификация по cookie-сессии, защита от некорректных входных данных и разграничение ролей на стороне интерфейса. Пароли не сохраняются в открытом виде: при создании пользователя выполняется bcrypt-хеширование, а при входе пароль проверяется сравнением с хешем.

В серверной сессии хранится только идентификатор пользователя. Дополнительная проверка в сервисах исключает создание пользователя с пустым именем или невалидным email, а также создание результата с отрицательным количеством баллов или отсутствующими связанными сущностями. Это показывает базовые принципы серверной безопасности.

Описание клиентской части

Клиентская часть приложения реализована на React с использованием Vite. Она представляет собой одностраничное приложение, в котором все основные действия выполняются в рамках одной рабочей области. Такой подход выбран для того, чтобы не усложнять реализацию большим количеством экранов.

Основным компонентом приложения является App. Он управляет общим состоянием интерфейса, хранит данные о текущем пользователе, списках пользователей, тестов и результатов, а также отвечает за загрузку данных после входа в систему. Вспомогательные части интерфейса вынесены в отдельные компоненты:

- LoginForm
- TestsList
- TestEditor
- ResultsTable
- UserForm

Для взаимодействия с серверной частью используется отдельный модуль ApiClient, который инкапсулирует HTTP-запросы. Это позволяет не смешивать интерфейсную логику и работу с API внутри компонентов.

Форма входа и блок текущего пользователя

До авторизации приложение показывает только форму входа. Пользователь вводит email и пароль, после чего данные отправляются на сервер. При успешной авторизации сервер создает сессию и возвращает сведения о текущем пользователе.

После входа тот же интерфейсный блок отображает уже имя пользователя, его роль и кнопку выхода из системы. Таким образом, форма входа и состояние авторизованного пользователя объединены в одном компоненте, что делает клиентскую часть компактнее и проще.

В localStorage сохраняется только последний введенный email. Пароль и основные данные приложения на клиенте не хранятся.

Результаты тестов
Учет тестов, студентов и итоговых баллов

Вход

Email
admin@example.com

Пароль
password

Войти

Демо: admin@example.com, maria.teacher@example.com, anna@example.com. Пароль: password.

Рисунок 5 – Форма входа в приложение

Сводная информация по системе

После успешного входа пользователь видит краткую сводку по текущим данным приложения. В ней отображаются:

- количество тестов;
- количество результатов;
- средний процент выполнения;
- количество зачтенных попыток.

Этот блок не является отдельной сущностью предметной области, но делает интерфейс удобнее для восприятия. Он позволяет быстро оценить текущее состояние системы без просмотра всех записей по отдельности.

Результаты тестов

Учет тестов, студентов и итоговых баллов

Вход

Администратор

Роль: admin

Выйти

Тестов: 2

Результатов: 3

Средний процент: 75%

Зачтено: 2

Тесты

Основы HTML и CSS
Веб-разработка
Проходной балл: 60%

Удалить

JavaScript и LocalStorage
Веб-разработка
Проходной балл: 70%

Удалить

Новый тест

Название

Предмет

Описание

Проходной процент

60

Сохранить

Результаты

Показаны все результаты

Студент	Тест	Балл	%	Оценка	Статус	Комментарий	
Анна Смирнова	Основы HTML и CSS	18/20	90%	5	зачтено	Отличный результат	Удалить
Игорь Петров	Основы HTML и CSS	11/20	55%	2	не зачтено	Нужно повторить тему	Удалить
Анна Смирнова	JavaScript и LocalStorage	16/20	80%	4	зачтено	Хорошее понимание API браузера	Удалить

Студент

Тест

Балл

Максимум

Комментарий

Анна Смирнова

Основы HTML и CSS

0

1

Добавить результат

Новый пользователь

Имя

Email

Пароль

Возраст

Роль

student

Создать

Рисунок 6 – Главная страница приложения.

Работа с тестами

Блок тестов состоит из двух частей: списка тестов и формы создания или редактирования теста.

Компонент TestsList отображает все тесты в виде списка. Для каждой записи выводятся название теста, предмет и проходной процент. При выборе теста его данные передаются в форму редактирования.

Компонент TestEditor используется для создания нового теста и изменения существующего. В форме указываются:

- название;
- предмет;
- описание;

- проходной процент.

Для ролей admin и teacher доступны операции создания, изменения и удаления тестов. Для роли student этот блок работает только в режиме просмотра.

С точки зрения пользовательского сценария этот раздел демонстрирует обычную работу с данными: получение списка, выбор записи, заполнение формы и отправку изменений на сервер.

Рисунок 7 – Страница управления тестами

Работа с результатами

Блок результатов реализован компонентом ResultsTable. Он содержит таблицу результатов и форму добавления новой записи.

В таблице для каждого результата выводятся:

- студент;
- тест;
- набранный балл;
- процент выполнения;
- итоговая оценка;
- статус зачета;
- комментарий.

Если в систему вошел студент, он видит только собственные результаты. Если вошел преподаватель или администратор, отображаются все результаты.

Ниже таблицы для пользователей с правами редактирования расположена форма добавления результата. В ней выбираются студент и тест, после чего вводятся балл, максимальный балл и комментарий. После отправки данных выполняется запрос к серверу, а интерфейс обновляется с учетом новых данных.

Здесь одновременно отображаются и исходные данные, и вычисляемые показатели тестирования.

Результаты

Показаны все результаты

Студент	Тест	Балл	%	Оценка	Статус	Комментарий	
Анна Смирнова	Основы HTML и CSS	18/20	90%	5	зачтено	Отличный результат	Удалить
Игорь Петров	Основы HTML и CSS	11/20	55%	2	не зачтено	Нужно повторить тему	Удалить
Анна Смирнова	JavaScript и LocalStorage	16/20	80%	4	зачтено	Хорошее понимание API браузера	Удалить

Студент

Тест

Балл

Максимум

Комментарий

Анна Смирнова

Основы HTML и CSS

0

1

Добавить результат

Рисунок 8 – Страница управления результатами

Создание пользователей

Создание новых пользователей реализовано отдельным компонентом UserForm. Он содержит поля:

- имя;
- email;
- пароль;
- возраст;
- роль.

После отправки формы выполняется запрос на создание пользователя через обязательный маршрут POST /users. Этот блок доступен только администратору, поскольку именно он отвечает за управление учетными записями.

На стороне клиента выполняется только базовая работа с формой. Основная валидация входных данных остается на серверной стороне, что позволяет не дублировать проверку в нескольких местах.

Новый пользователь					
Имя	Email	Пароль	Возраст	Роль	
				student	Создать

Рисунок 9 – Страница пользователей

Организация состояния и ограничений доступа

Состояние клиентской части хранится в компоненте App. В нем поддерживаются сведения о текущем пользователе, загруженных списках данных, выбранном тесте, а также состояниях загрузки и ошибки.

На клиенте также реализованы интерфейсные ограничения по ролям:

- студент не видит кнопки редактирования и удаления;

- преподаватель и администратор могут управлять тестами и результатами;
- только администратор может создавать новых пользователей.

Эти ограничения не заменяют серверную проверку прав, но делают интерфейс понятнее и не показывают пользователю действия, которые ему все равно недоступны.

Заключение

В рамках курсовой работы было разработано клиент-серверное web-приложение Test Results для учета результатов тестирования.

Предметная область – учебная система, в которой требуется хранить тесты, пользователей и результаты прохождения. Рассматриваемая проблема – отсутствие единого структурированного инструмента для учета результатов, автоматического расчета оценки и разграничения доступа.

Для решения задачи была спроектирована реляционная база данных PostgreSQL, включающая таблицы users, tests, questions, answers, test_results и result_audit, а также представления, функции, процедуры и триггеры. Эта схема оформлена отдельно в каталоге DB. Серверная часть реализована на FastAPI и организована по слоям routers – services – storage, где хранилищем выступает память процесса. Клиентская часть реализована на React с использованием Vite.

В результате выполнения работы получено функционирующее веб-приложение, удовлетворяющее требованиям: реализованы CRUD-операции для двух и более сущностей, подготовлены объекты базы данных, обеспечено взаимодействие клиента и сервера через REST API, предусмотрены авторизация и разграничение доступа по ролям, логирование. Проект объединяет все основные темы курса, поэтому приложение подходит так же как учебный пример клиент-серверной системы.

Список литературы

1. ГОСТ 7.32-2017. Отчет о научно-исследовательской работе. Структура и правила оформления.
2. React. Documentation [Электронный ресурс]. URL: <https://react.dev/>.
3. FastAPI. Documentation [Электронный ресурс]. URL: <https://fastapi.tiangolo.com/>.
4. PostgreSQL Documentation [Электронный ресурс]. URL: <https://www.postgresql.org/docs/>.
5. MDN Web Docs. Fetch API [Электронный ресурс]. URL: https://developer.mozilla.org/docs/Web/API/Fetch_API.
6. bcrypt. Documentation [Электронный ресурс]. URL: <https://pypi.org/project/bcrypt/>.
7. Starlette. SessionMiddleware [Электронный ресурс]. URL: <https://www.starlette.io/middleware/#sessionmiddleware>.
8. Исходный код проекта Test Results [Электронный ресурс]. URL: <https://github.com/avolxn/csa-coursework.git>.